

# Branch-and-Price

Tulay Flamand

Business School

University of Colorado Denver

# Motivation

- Branch-and-price approach is developed based on branch-and-bound framework.
- Basically, branch-and-bound solve a sequence of LP relaxations of a given IP.
- Branch-and-bound/cut is a generalized form of branch-and-bound, which tightens the LP relaxations by adding cuts (rows).
- Branch-and-price is a generalized form of branch-and-bound, which tightens the LP relaxation by generating and adding favorable columns.
- If the methodology is generalized to add rows and columns together, it is called branch-and-price-and-cut.
- These columns are generated in a decomposition methodology, by iteratively solving sub-problems or **pricing problems**.

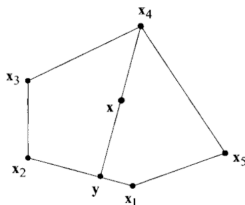
# Motivation

- Dantzig-Wolfe (D-W) decomposition is used to tackle problems that would otherwise be too large or too difficult to solve.
- The strategy of the decomposition framework is to consider two separate linear programs, namely master problem and sub-problem(s).
- The master problem includes the set of general constraints.
- There are smaller sub-problems that can be solved independently, or possess a special structure (such as a network structure), allowing a quicker solution.
- The master problem passes down a continually revised set of cost coefficients (prices) to the sub-problem, and receives from the sub-problem a new column(s) based on these cost coefficients. Hence, it is also known as a **column generation (CG)** approach.

# Quick Historical Perspective for the CG

- First introduced by G.B. Dantzig & P. Wolfe, Decomposition Principle for Linear Programs. Oper. Res. 8, 101-111. (1960)
- Popularized by P.C. Gilmore & R.E. Gomory, A Linear Programming Approach to the Cutting Stock Problem. Oper. Res. 9, 849-859. (1960)

# Dantzig-Wolfe Decomposition



$$\mathbf{x} = \lambda \mathbf{y} + (1 - \lambda) \mathbf{x}_4, \quad \text{where } \lambda \in (0, 1).$$

$$\mathbf{y} = \mu \mathbf{x}_1 + (1 - \mu) \mathbf{x}_2, \quad \text{where } \mu \in (0, 1).$$

$$\mathbf{x} = \lambda \mu \mathbf{x}_1 + \lambda(1 - \mu) \mathbf{x}_2 + (1 - \lambda) \mathbf{x}_4.$$

$$\lambda \mu + \lambda(1 - \mu) + (1 - \lambda) = 1.$$

On  $\text{convex}(X)$ , define set:

- $\Omega$ : set of all extreme points

$\mathbf{x} \in \text{convex}(X)$  rewritten as

$$\mathbf{x} = \sum_{p \in \Omega} \mathbf{x}_p \lambda_p \quad (1a)$$

$$\sum_{p \in \Omega} \lambda_p = 1 \quad (1b)$$

$$\lambda_p \geq 0, \forall p \in \Omega \quad (1c)$$

$$(1d)$$

# Dantzig-Wolfe Decomposition

$$\begin{aligned}
 \mathbf{P}: \text{Minimize } & cx & (2a) \\
 \text{subject to } & Ax = b & (2b) \\
 & x \in X \equiv \{x : 0 \leq x \leq UB\} & (2c)
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{P}: \text{Minimize } & c\left(\sum_{p \in \Omega} x_p \lambda_p\right) & (3a) \\
 & A\left(\sum_{p \in \Omega} x_p \lambda_p\right) = b & (3b) \\
 & \sum_{p \in \Omega} \lambda_p = 1 & (3c) \\
 & \lambda_p \geq 0, \forall p \in \Omega & (3d) \\
 & & (3e)
 \end{aligned}$$

# Dantzig-Wolfe Decomposition

$$\begin{aligned}
 \mathbf{P}: \text{Minimize } & cx & (2a) \\
 \text{subject to } & Ax = b & (2b) \\
 & x \in X \equiv \{x : 0 \leq x \leq UB\} & (2c)
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{P}: \text{Minimize } & c\left(\sum_{p \in \Omega} x_p \lambda_p\right) & (3a) \\
 & A\left(\sum_{p \in \Omega} x_p \lambda_p\right) = b & (3b) \\
 & \sum_{p \in \Omega} \lambda_p = 1 & (3c) \\
 & \lambda_p \geq 0, \forall p \in \Omega & (3d) \\
 & & (3e)
 \end{aligned}$$

# Dantzig-Wolfe Decomposition - The Master Problem

$$\mathbf{P}: \text{Minimize } c\left(\sum_{p \in \Omega} x_p \lambda_p\right) \quad (4a)$$

$$A\left(\sum_{p \in \Omega} x_p \lambda_p\right) = b \quad (4b)$$

$$\sum_{p \in \Omega} \lambda_p = 1 \quad (4c)$$

$$\lambda_p \geq 0, \forall p \in \Omega \quad (4d)$$

$$(4e)$$

$$\mathbf{P}: \text{Minimize } \sum_{p \in \Omega} (cx_p) \lambda_p \quad (5a)$$

$$\sum_{p \in \Omega} (Ax_p) \lambda_p = b \quad (5b)$$

$$\sum_{p \in \Omega} \lambda_p = 1 \quad (5c)$$

$$\lambda_p \geq 0, \forall p \in \Omega \quad (5d)$$

$$(5e)$$

There may be too many extreme points, where the total enumeration of all may be difficult. Instead, we will solve a **restricted** master problem (RMP) with a limited number of extreme points. At each iteration, we will generate a new favorable extreme point, and add it to the master problem to be resolved.



# Dantzig-Wolfe Decomposition - The Restricted Master Problem

- $\Omega' \subset \Omega$ : set of some extreme points

$$\textbf{RMP: Minimize} \quad \sum_{p \in \Omega'} (cx_p)\lambda_p \quad (6a)$$

$$\sum_{p \in \Omega'} (Ax_p)\lambda_p = b \quad \leftarrow \pi \quad (6b)$$

$$\sum_{p \in \Omega'} \lambda_p = 1 \quad \leftarrow \pi_0 \quad (6c)$$

$$\lambda_p \geq 0, \forall p \in \Omega' \quad (6d)$$

(6e)

# Dantzig-Wolfe Decomposition

- In pricing step of simplex algorithm, one basic variable leaves and one non-basic variable enters the basis.
- This decision is made from the reduced cost vector  $c_N - c_B B^{-1} N$  or  $c_N - \pi N$ .
- The component with most negative value determines the entering variable.
- In terms of columns, we need to find:  $\arg \min_j \{c_j - \pi a_j \mid a_j \text{ is a column of } N\}$ .
- We need to solve a pricing sub-problem.

# Dantzig-Wolfe Decomposition - Sub-problem

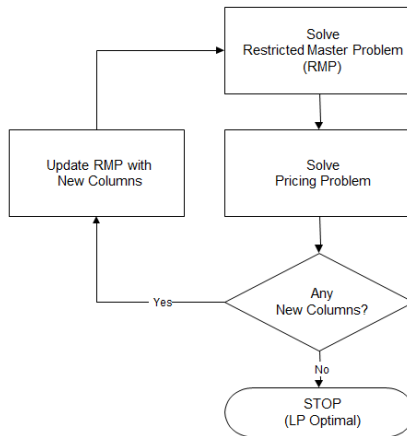
- Given the current dual multipliers for a subset of columns:
  - $\pi$  for coupling constraints
  - $\pi_0$  for convexity constraint
- generate (if possible) new columns with negative reduced cost:

$$\mathbf{SP:} \text{ Minimize } cx - \pi(Ax) - \pi_0 \quad (7a)$$

$$x \in \text{convex}(X) \quad (7b)$$

- $x_p$  is an extreme point if  $cx_p - \pi(Ax_p) - \pi_0 < 0$ , for some  $p \in \Omega$ . We should add  $x_p$  to  $\Omega'$  and resolve the RMP.

# Column Generation



# Column Generation

- Column Generation corresponds to the solution process used in the Dantzig-Wolfe decomposition.
- This approach can also be used directly by formulating a master problem and sub-problems rather than obtaining them by decomposing a global formulation of the problem.

# Column Generation

- Column Generation approach terminates when there is no profitable columns which can be generated with a negative reduced cost.
- Column Generation finds an optimal solution to the LP. Hence, it generates a dual bound for an integer problem (lower bound for a minimization problem, upper bound for a maximization problem).

# Generalized Assignment Problem (GAP)

- There are  $n$  tasks to be assigned to  $m$  machines with different capacities. Each task should be assigned to only one machine.  $w_{ij}$  represents how much of the capacity of machine  $j$  is consumed by task  $i$ . The objective is to find a maximum profit assignment of tasks to machines subject to capacity restrictions of the machines.
- $\mathcal{M}$  : Set of machines.
- $\mathcal{N}$  : Set of tasks.
- $p_{ij}$  : The profit of assigning task  $i$  to machine  $j$ ,  $\forall i \in \mathcal{N}, j \in \mathcal{M}$ .
- $d_j$  : Capacity of machine  $j$ ,  $\forall j \in \mathcal{M}$ .
- $y_{ij}$  : Binary variable, 1 if task  $i$  is assigned to machine  $j$ , 0 otherwise,  $\forall i \in \mathcal{N}, j \in \mathcal{M}$ .

$$\mathbf{P}: \text{Maximize } \sum_{i \in \mathcal{N}} \sum_{j \in \mathcal{M}} p_{ij} y_{ij} \quad (8a)$$

$$\sum_{j \in \mathcal{M}} y_{ij} = 1, \forall i \in \mathcal{N} \quad (8b)$$

$$\sum_{i \in \mathcal{N}} w_{ij} y_{ij} \leq d_j, \forall j \in \mathcal{M} \quad (8c)$$

$$y \text{ binary.} \quad (8d)$$

# CG Formulation of GAP

$$a_{jh} = \begin{matrix} i=1 \\ i=2 \\ \vdots \\ i \\ \vdots \\ i=|\mathcal{N}| \end{matrix} \begin{bmatrix} (a_{jh}) \end{bmatrix}, \text{ where } a_{jh}^i = \begin{cases} 1, & \text{if task } i \text{ is included in column } h \text{ of machine } i, \forall i \in \mathcal{N} \\ 0, & \text{otherwise.} \end{cases}$$

- Hence, GAP can be formulated as a set partitioning model.
- $\mathcal{C}_j$  : All feasible patterns (columns) for machine  $j$ ,  $\forall j \in \mathcal{M}$ .
- $\delta_{jh}$  : The reward of  $h$ th column of machine  $j$ ,  $\forall j \in \mathcal{M}$ ,  $h \in \mathcal{C}_j$ .
- $x_{jh}$  : Binary variable, 1 if  $h$ th column of machine  $j$  is chosen, 0 otherwise,  $\forall j \in \mathcal{M}$ ,  $h \in \mathcal{C}_j$ .

$$\mathbf{P}: \text{Maximize } \sum_{j \in \mathcal{M}} \sum_{h \in \mathcal{C}_j} \delta_{jh} x_{jh} \quad (9a)$$

$$\sum_{j \in \mathcal{M}} \sum_{h \in \mathcal{C}_j} a_{jh}^i x_{jh} = 1, \forall i \in \mathcal{N} \quad \leftarrow \pi_i \quad (9b)$$

$$\sum_{h \in \mathcal{C}_j} x_{jh} = 1, \forall j \in \mathcal{M} \quad \leftarrow \mu_j \quad (9c)$$

$$x \geq 0 \& \text{ integer} \quad (9d)$$



# The Restricted Master Problem for GAP

- $\mathcal{C}'_j \subset \mathcal{C}_j$  : set of **some** feasible patterns for machine  $j$ .

$$\mathbf{RMP:} \text{ Maximize } \sum_{j \in \mathcal{M}} \sum_{h \in \mathcal{C}'_j} \delta_{jh} x_{jh} \quad (10a)$$

$$\sum_{j \in \mathcal{M}} \sum_{h \in \mathcal{C}'_j} a_{jh}^i x_{jh} = 1, \forall i \in \mathcal{N} \quad \leftarrow \pi_i \quad (10b)$$

$$\sum_{h \in \mathcal{C}'_j} x_{jh} = 1, \forall j \in \mathcal{M} \quad \leftarrow \mu_j \quad (10c)$$

$$x \geq 0 \quad (10d)$$

# The Pricing Problem for GAP

- $z_i$  : Binary variable 1, if task  $i$  is chosen, 0 otherwise,  $\forall i \in \mathcal{N}$ .

$$\mathbf{SP}(j, \bar{\pi}, \bar{\mu}): \text{Maximize } \sum_{i \in \mathcal{N}} (p_{ij} z_i - \bar{\pi}_i z_i) - \mu_j \quad (11a)$$

$$\sum_{i \in \mathcal{N}} w_{ij} z_i \leq d_j \quad (11b)$$

$$z \text{ binary.} \quad (11c)$$

# Example for GAP

- Consider a GAP for two machines and three tasks. Profit, weight, and capacity information are given as follows:

| $p_{ij}$      | Machine 1 | Machine 2 |
|---------------|-----------|-----------|
| <b>Task 1</b> | 1         | 4         |
| <b>Task 2</b> | 3         | 5         |
| <b>Task 3</b> | 8         | 10        |

| $w_{ij}$      | Machine 1 | Machine 2 |
|---------------|-----------|-----------|
| <b>Task 1</b> | 4         | 3         |
| <b>Task 2</b> | 5         | 5         |
| <b>Task 3</b> | 3         | 2         |

| $d_j$ | Machine 1 | Machine 2 |
|-------|-----------|-----------|
|       | 8         | 5         |

- Let's start with some initial columns:

| $c_j$         | (1,1) | (2,1) |
|---------------|-------|-------|
| <b>Task 1</b> | 1     | 0     |
| <b>Task 2</b> | 0     | 1     |
| <b>Task 3</b> | 1     | 0     |
| $\delta_{jh}$ | 9     | 5     |

# Example for GAP

- Let's start with some initial columns:

| $C_j$         | (1,1) | (2,1) |
|---------------|-------|-------|
| <b>Task 1</b> | 1     | 0     |
| <b>Task 2</b> | 0     | 1     |
| <b>Task 3</b> | 1     | 0     |
| $\delta_{jh}$ | 9     | 5     |

$$\text{RMP: Maximize } 9x_{11} + 5x_{21} \quad (12a)$$

$$x_{11} = 1 \quad (12b)$$

$$x_{21} = 1 \quad (12c)$$

$$x_{11} = 1 \quad (12d)$$

$$x_{11} = 1 \quad (12e)$$

$$x_{21} = 1 \quad (12f)$$

$$x \geq 0 \quad (12g)$$

- $\bar{\pi}_i = 0, \forall i \in \mathcal{N}, \mu_j = 0, \forall j \in \mathcal{M}.$

# Example for GAP

| $p_{ij}$ | Machine 1 | Machine 2 |
|----------|-----------|-----------|
| Task 1   | 1         | 4         |
| Task 2   | 3         | 5         |
| Task 3   | 8         | 10        |

| $w_{ij}$ | Machine 1 | Machine 2 |
|----------|-----------|-----------|
| Task 1   | 4         | 3         |
| Task 2   | 5         | 5         |
| Task 3   | 3         | 2         |

| $d_j$ | Machine 1 | Machine 2 |
|-------|-----------|-----------|
|       | 8         | 5         |

$$\text{SP}(1, \bar{\pi}, \bar{\mu}): \text{Maximize } z_1 + 3z_2 + 8z_3 \quad (13a)$$

$$4z_1 + 5z_2 + 3z_3 \leq 8 \quad (13b)$$

$$z \text{ binary.} \quad (13c)$$

$$\text{SP}(2, \bar{\pi}, \bar{\mu}): \text{Maximize } 4z_1 + 5z_2 + 10z_3 \quad (14a)$$

$$3z_1 + 5z_2 + 2z_3 \leq 5 \quad (14b)$$

$$z \text{ binary.} \quad (14c)$$

# Example for GAP

| $C_j$         | (1,1) | (2,1) | (1,2) | (2,2) |
|---------------|-------|-------|-------|-------|
| <b>Task 1</b> | 1     | 0     | 0     | 1     |
| <b>Task 2</b> | 0     | 1     | 1     | 0     |
| <b>Task 3</b> | 1     | 0     | 1     | 1     |
| $\delta_{jh}$ | 9     | 5     | 11    | 14    |

$$\text{RMP: Maximize } 9x_{11} + 5x_{21} + 11x_{12} + 14x_{22} \quad (15a)$$

$$x_{11} + x_{22} = 1 \quad (15b)$$

$$x_{21} + x_{12} = 1 \quad (15c)$$

$$x_{11} + x_{12} + x_{22} = 1 \quad (15d)$$

$$x_{11} + x_{12} = 1 \quad (15e)$$

$$x_{21} + x_{22} = 1 \quad (15f)$$

$$x \geq 0 \quad (15g)$$

•  $\bar{\pi}_1 = 3, \bar{\pi}_2 = 5, \bar{\pi}_3 = 11, \mu_1 = -5, \mu_2 = 0.$

# Example for GAP

$$\mathbf{SP}(1, \bar{\pi}, \bar{\mu}): \text{Maximize } z_1 + 3z_2 + 8z_3 - (3z_1 + 5z_2 + 11z_3) + 5 \quad (16a)$$

$$4z_1 + 5z_2 + 3z_3 \leq 8 \quad (16b)$$

$$z \text{ binary.} \quad (16c)$$

$$\mathbf{SP}(2, \bar{\pi}, \bar{\mu}): \text{Maximize } 4z_1 + 5z_2 + 10z_3 - (3z_1 + 5z_2 + 11z_3) - 0 \quad (17a)$$

$$3z_1 + 5z_2 + 2z_3 \leq 5 \quad (17b)$$

$$z \text{ binary.} \quad (17c)$$

# Example for GAP

- Iterations continue until no favorable columns are found. At the end of all iterations, following are the columns added to the problem:

| $C_j$         | (1,1) | (2,1) | (1,2) | (2,2) | (1,3) | (2,3) | (1,4) | (1,5) |
|---------------|-------|-------|-------|-------|-------|-------|-------|-------|
| <b>Task 1</b> | 1     | 0     | 0     | 1     | 0     | 1     | 1     | 0     |
| <b>Task 2</b> | 0     | 1     | 1     | 0     | 0     | 0     | 0     | 1     |
| <b>Task 3</b> | 1     | 0     | 1     | 1     | 0     | 0     | 0     | 0     |
| $\delta_{jh}$ | 9     | 5     | 11    | 14    | 0     | 4     | 1     | 3     |

- Optimal solution of RMP is:  $x_{15} = 1$ ,  $x_{22} = 1$ , and  $z^* = 17$ .
- Here, the LP solution is integral, therefore it is also optimal for the integer problem. What if the solution was fractional?
- Rounding?
- Solving the RMP as an integer problem?
- Would these guarantee optimality?



# Variable Selection

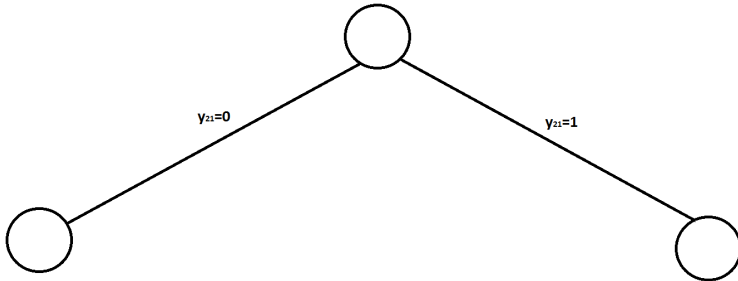
- Now suppose the solution of the problem is  $x_{12} = 0.7$ ,  $x_{15} = 0.3$ ,  $x_{22} = 0.3$ ,  $x_{23} = 0.7$ .
- We need to branch!
- In branch-and-price scheme, standard branching on the  $x$  variable creates a problem along a branch, where a variable has been set to 0. For example, if we have a branch with a branching constraint  $x_{12} = 0$ , this constraint can be added to the RMP.
- However, most likely, the next time a sub-problem is solved for machine 1, the same column will be created again, because the sub-problem does not consider  $x$  variables while creating columns. Hence, we need to force the sub-problem to generate the next best column, which is very hard.
- For this reason, in branch-and-price scheme, usually original variables are selected to branch on.

# Variable Selection

- Select an integer variable that is fractional in LP solution.
- Usually, need to select one from a set  $C$  of variables that are taking fractional values.
- There is no specific rule for how to select a variable.
- Common to select “most fractional variable” (MFV).
- $\arg \max_{j \in C} \min \{f_j, 1 - f_j\}$ , where  $f_j = \bar{x} - \lfloor \bar{x}_j \rfloor$ .
- For example, for  $x_{12}$  and  $x_{23}$ , this value is  $\min \{(0.7 - 0), 0.3\} = 0.3$ , and for  $x_{12}$  and  $x_{23}$ , it is  $\min \{(0.3 - 0), 0.7\} = 0.3$ . Therefore, we can select any of them.
- Suppose that we select  $x_{12}$ , then we need to follow a scheme to select an original variable to branch on.
- For example, one can select an original variable which is related to a task assigned to the selected column with the highest weight  $w$  parameter.
- $x_{12}$  includes Task 2 and Task 3 for Machine 1. Among them, Task 2 has the highest weight. Then one can select original variable  $y_{21}$  to branch on such that  $y_{21} = 0$  and  $y_{21} = 1$ .

# Variable Selection

| (1,1) | (2,1) | (1,2) | (2,2) | (1,3) | (2,3) | (1,4) | (1,5) |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 1     | 0     | 0     | 1     | 0     | 1     | 1     | 0     |
| 0     | 1     | 1     | 0     | 0     | 0     | 0     | 1     |
| 1     | 0     | 1     | 1     | 0     | 0     | 0     | 0     |



| (1,1) | (2,1) | (2,2) | (1,3) | (2,3) | (1,4) |
|-------|-------|-------|-------|-------|-------|
| 1     | 0     | 1     | 0     | 1     | 1     |
| 0     | 1     | 0     | 0     | 0     | 0     |
| 1     | 0     | 1     | 0     | 0     | 0     |

| (2,1) | (1,2) | (2,2) | (2,3) | (1,5) |
|-------|-------|-------|-------|-------|
| 0     | 0     | 1     | 1     | 0     |
| 1     | 1     | 0     | 0     | 1     |
| 0     | 1     | 1     | 0     | 0     |

# The Pricing Problem for GAP for the Branch $y_{21} = 0$

$$\mathbf{SP}(1, \bar{\pi}, \bar{\mu}): \text{Maximize } \sum_{i \in \mathcal{N}} (p_{ij} z_i - \bar{\pi}_i z_i) - \mu_j \quad (18a)$$

$$\sum_{i \in \mathcal{N}} w_{ij} z_i \leq d_j \quad (18b)$$

$$z_2 = 0 \quad (18c)$$

$$z \text{ binary.} \quad (18d)$$

# Node Selection

## Depth-First Strategy

- Keep descending until you fathom a node, then backtrack and descend again.
- Identify a primal bound as soon as possible. This will help in turn eliminate unpromising nodes.
- Easy to re-optimize an LP problem when a simple constraint gets added.
- Memory friendly: less number of active nodes.

## Best-First Strategy

- Select among active nodes one that has the best dual bound (upper bound for maximization, lower bound for minimization).
- Exploring nodes that seem to have the highest potential.
- Memory intensive: usually induces a proliferation of active nodes.

# Performance of the Branch-and-Price for GAP with Location/Allocation Constraints

| Data Set | $( B ,  N )$ | $ K_s $ | Instance    | CPLEX/Model MKP |               |             | B&P Algorithm |             |              |                |
|----------|--------------|---------|-------------|-----------------|---------------|-------------|---------------|-------------|--------------|----------------|
|          |              |         |             | CPU(s)          | B&P Nodes     | Gap(%)      | CPU(s)        | B&P Nodes   | Gap (%)      | CPU Savings(%) |
| Set 1    | (5,15)       | 5-8     | 1           | 98.5            | 17859         | 0           | 19.6          | 1           | 0            | 80.1           |
|          |              |         | 2           | 269.1           | 91786         | 0           | 13.6          | 1           | 0            | 94.9           |
|          |              |         | 3           | 80.7            | 30909         | 0           | 15.4          | 1           | 0            | 80.9           |
|          |              |         | 4           | 224.9           | 49244         | 0           | 12.5          | 1           | 0            | 94.4           |
|          |              |         | 5           | 516.6           | 126379        | 0           | 33.6          | 1           | 0            | 93.5           |
|          |              |         | <b>Avr.</b> | <b>237.9</b>    | <b>63235</b>  | <b>0</b>    | <b>18.9</b>   | <b>1</b>    | <b>0</b>     | <b>88.7</b>    |
| Set 2    | (5,15)       | 7-10    | 1           | 1131.3          | 139603        | 0           | 17.2          | 1           | 0            | 98.4           |
|          |              |         | 2           | 547.7           | 70176         | 0           | 28.2          | 1           | 0            | 94.8           |
|          |              |         | 3           | 787.3           | 79492         | 0           | 23.4          | 1           | 0            | 97             |
|          |              |         | 4           | 1680.1          | 182307        | 0           | 41.8          | 3           | 0            | 97.5           |
|          |              |         | 5           | 1749.8          | 148294        | 0           | 31.6          | 1           | 0            | 98.1           |
|          |              |         | <b>Avr.</b> | <b>1179.2</b>   | <b>123974</b> | <b>0</b>    | <b>28.4</b>   | <b>1.4</b>  | <b>0</b>     | <b>97.2</b>    |
| Set 3    | (5,15)       | 6-9     | 1           | 1548.5          | 227013        | 0           | 39.3          | 1           | 0            | 97.4           |
|          |              |         | 2           | 1215.1          | 220356        | 0           | 24.1          | 1           | 0            | 98             |
|          |              |         | 3           | 431.2           | 125927        | 0           | 33.9          | 1           | 0            | 92.1           |
|          |              |         | 4           | 499.9           | 147994        | 0           | 29.7          | 1           | 0            | 94             |
|          |              |         | 5           | 502.1           | 95319         | 0           | 154.1         | 49          | 0            | 69.3           |
|          |              |         | <b>Avr.</b> | <b>839.3</b>    | <b>163321</b> | <b>0</b>    | <b>56.2</b>   | <b>10.6</b> | <b>0</b>     | <b>90.2</b>    |
| Set 4    | (5,15)       | 7-8     | 1           | 396.6           | 63845         | 0           | 36.1          | 5           | 0            | 90.9           |
|          |              |         | 2           | 156.3           | 35690         | 0           | 14.7          | 1           | 0            | 90.6           |
|          |              |         | 3           | 220.6           | 40370         | 0           | 17.7          | 1           | 0            | 91.9           |
|          |              |         | 4           | 358.1           | 66223         | 0           | 26.8          | 5           | 0            | 92.5           |
|          |              |         | 5           | 347.3           | 69802         | 0           | 17.5          | 1           | 0            | 94.9           |
|          |              |         | <b>Avr.</b> | <b>295.7</b>    | <b>55186</b>  | <b>0</b>    | <b>22.5</b>   | <b>2.6</b>  | <b>0</b>     | <b>92.1</b>    |
| Set 5    | (5,15)       | 7-8     | 1           | 3600            | 331675        | 0.1         | 53            | 5           | 0            | 98.5           |
|          |              |         | 2           | 622.4           | 131823        | 0           | 21.4          | 1           | 0            | 96.5           |
|          |              |         | 3           | 138.6           | 30820         | 0           | 20.7          | 3           | 0            | 85             |
|          |              |         | 4           | 167.3           | 39221         | 0           | 70.4          | 17          | 0            | 57.9           |
|          |              |         | 5           | 1035.3          | 184036        | 0           | 23.4          | 1           | 0            | 97.7           |
|          |              |         | <b>Avr.</b> | <b>1112.7</b>   | <b>143515</b> | <b>0.02</b> | <b>37.7</b>   | <b>5.4</b>  | <b>0</b>     | <b>87.1</b>    |
| Set 6    | (5,20)       | 8-11    | 1           | 3600            | 150429        | 0.2         | 723.2         | 21          | 0            | 79.9           |
|          |              |         | 2           | 3600            | 167866        | 0.03        | 427.6         | 19          | 0            | 88.1           |
|          |              |         | 3           | 3600            | 127321        | 0.2         | 241.2         | 5           | 0            | 93.3           |
|          |              |         | 4           | 3600            | 55265         | 0.09        | 371.8         | 25          | 0            | 89.6           |
|          |              |         | 5           | 3600            | 164804        | 0.5         | 263.9         | 3           | 0            | 92.6           |
|          |              |         | <b>Avr.</b> | <b>3600</b>     | <b>133137</b> | <b>0.2</b>  | <b>401</b>    | <b>14.2</b> | <b>0</b>     | <b>88.7</b>    |
| Set 7    | (5,20)       | 9-10    | 1           | 3600            | 149545        | 0.1         | 126.7         | 1           | 0            | 96.4           |
|          |              |         | 2           | 3600            | 198124        | 0.6         | 648.2         | 25          | 0            | 81.9           |
|          |              |         | 3           | 3600            | 144481        | 0.8         | 3600          | 219         | 0.02         | -              |
|          |              |         | 4           | 3600            | 107268        | 0.4         | 182.4         | 1           | 0            | 94.9           |
|          |              |         | 5           | 3600            | 151226        | 0.7         | 3600          | 99          | 0            | -              |
|          |              |         | <b>Avr.</b> | <b>3600</b>     | <b>150128</b> | <b>0.5</b>  | <b>1631.4</b> | <b>69</b>   | <b>0.004</b> | <b>91.1</b>    |

# References

- Bazaara S, Jarvis JJ, Sherali HD (2010). Linear Programming and Network Flows, 4th Edition, John Wiley & Sons Inc., New York.
- Chen DS, Batson RG, Dang Yu (2010). Applied Integer Programming: Modeling and Solution, 1st Edition, John Wiley & Sons Inc., New York.
- Daruka GP, Palekar US (2005). Application of Column Generation Techniques to Retail Assortment Planning. In: Integer Programming Theory and Practice, edited by Karlof J., Taylor & Francis.
- Savelsbergh M (1997). A Branch-and-Price Algorithm for the Generalized Assignment Problem, Operations Research, 45(6):831–841.
- Ghoniem A, Flamand T, Haouari M (2016). Exact Solution Methods for a Generalized Assignment Problem with Location/Allocation Considerations, INFORMS Journal on Computing, 28(3): 589–602.

The background of the slide is a repeating pattern of colorful 3D question marks in shades of blue, green, yellow, pink, and purple. The text is centered in the middle of the slide.

**THANK YOU...**

**QUESTIONS?**